# Engineering Agentic Systems: Toward Verification-First AIware Engineering

Qinghua Lu, CSIRO
Ahmed E. Hassan, Queen's University
David Lo, Singapore Management University
Thomas Zimmermann, University of California, Irvine

// *Agentic systems are transforming software engineering from a code-centric discipline toward AIware engineering, where intelligence is deliberately allocated across foundation models, on-demand generated software, and external tools. The central challenge is no longer generating behaviours, but verifying them at scale. Verification-Driven AIware Engineering addresses this challenge by treating verifiability as a first-class design objective, deliberately decomposing problems to enable scalable verification.* //

*Digital Object Identifier 10.1109/MS.2026.3697911*
*Date of current version: 8 September 2026*



0740-7459 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies.

**AN AGENTIC SYSTEM** is a foundation model-based AI system that integrates multimodal context management, memory, reasoning, external tools and agents, and autonomous feedback loops to pursue goals autonomously and adapt its behavior over time. When engineering agentic systems, it is important to move beyond a narrow model-centric perspective and instead adopt a system-level view. Foundation models, such as large language models (LLMs), are only one component within a broader AI system. Other components, such as memory, external tools, verifiers, and guardrails, interact with LLMs to collectively shape system behavior and capabilities. Moreover, agentic systems often involve multiple interacting agents, humans, and tools operating in dynamic and evolving open world environments. Building agentic systems therefore requires rigorous system-level AI engineering principles and techniques to ensure critical qualities, such as robustness, safety, and observability.[1]

## AIware Engineering: A New Paradigm in AI Engineering

As illustrated in Figure 1, within AI engineering, we can distinguish two fundamentally different paradigms: AI-assisted software engineering and AIware engineering. The first paradigm is AI-assisted software engineering. In this paradigm, system behavior is largely prespecified in code, while AI augments the software development process by generating, testing, analyzing, and improving code artifacts. Although AI contributes to implementation, the overall system behavior remains primarily code-centric and explicitly defined by developers.

The second paradigm is AIware engineering. Within the software engineering research community, the term "AIware" is often used broadly to refer to AI-powered software, particularly foundation model-based software.[2] Here, however, we adopt a narrower definition to clearly differentiate AIware engineering from AI-assisted software engineering.

Traditional software engineering treated software code as the primary carrier of system behavior. AI agents shift increasing amounts of behavior into runtime goal-directed reasoning, planning, and interaction, which introduces autonomy and flexibility but weakens predictability, auditability, and repeatability. Deterministic software remains valuable because it offers stable execution, explicit checks, and reliable primitives. The future system is neither pure software nor pure agent, but a composition of both, which we refer to as AIware.

However, this shift also challenges many traditional engineering assumptions and abstractions. Concepts, such as "design time," "runtime," and even the "engineering lifecycle," become increasingly blurred when system behavior can emerge from prompts, context retrieval, orchestration loops, external tools, and potentially future forms of self-improvement. If an agent's behavior can be substantially reshaped through runtime feedback loops, retrieval augmentation, scaffold updates, or even model weight modification, it becomes less clear where system design begins and ends, and whether these changes should be viewed as design-time engineering or runtime adaptation. In this setting, engineering may no longer map cleanly onto fixed lifecycle stages, but instead resemble a continuous process of intelligence allocation,

> AIware engineering can be viewed as the deliberate allocation of intelligence across models, on-demand generated software, and external tools.

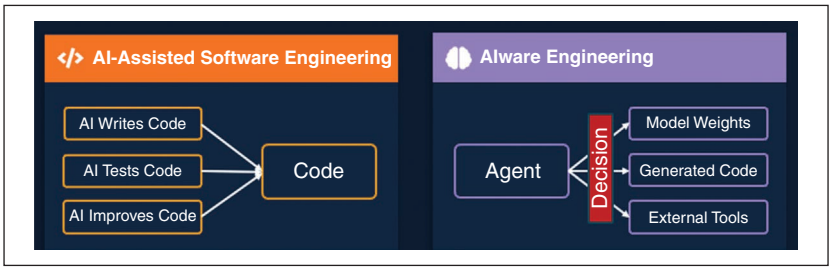



**FIGURE 1.** Two paradigms in AI engineering: AI-assisted software engineering and AIware engineering.

verification, and adaptation through autonomous feedback loops. In many agentic systems, runtime increasingly resembles a continuous experimentation and optimization layer rather than simple execution. Retrieval updates, prompt modifications, verifier feedback, memory evolution, orchestration policy changes, and adaptive tool selection may continuously reshape behavior after deployment. As a result, the distinction between inference-time adaptation and post-training optimization may become architecturally blurred rather than merely operationally separated.

From this perspective, AIware engineering can be viewed as the deliberate allocation of intelligence across models, on-demand generated software, and external tools, while recognizing that these allocations may continuously evolve rather than remain fixed after deployment. The central engineering problem is therefore not whether an AI agent should act directly or write software, but how different forms of intelligence should be allocated, verified, externalized, and progressively internalized over time. Determining where intelligence should reside becomes a fundamental architectural decision that directly affects critical system qualities, such as observability, verifiability, and controllability.

## Scalable Verification in AIware Engineering

Three major domains of AIware systems are beginning to emerge. The first is AI coding, where AI systems autonomously generate, test, and refine software. The second is AI cowork, where AI collaborates with humans to plan, coordinate, and execute complex workflows. The third is AI coscientist, where AI systems generate hypotheses, design experiments, and propose solutions to accelerate scientific discovery.

Software engineering pioneered scalable verification thinking. Traditional software engineering workflows evolved around three tightly connected activities: requirement specification, implementation, and verification through testing and analysis. Over decades, the field developed mature infrastructures for scalable verification, including specifications, benchmarks, regression testing pipelines, coverage analysis, testing environments, static analysis tools, and continuous integration systems. Dynamic verification and testing have already been heavily automated for a long time.

This mature verification infrastructure makes software particularly well suited to scalable solve-verify loops. It is therefore unsurprising that coding became the first domain to be rapidly automated by agentic AI systems. Code is not only generatable, but also highly verifiable. In contrast, domains such as AI cowork and AI coscientist contain verification artifacts that are often incomplete, weakly structured, or inherently dependent on human judgment. Tasks that can be automatically checked are deeply intertwined with tasks requiring contextual interpretation, tacit knowledge, or scientific reasoning. As a result, human involvement in these domains remains substantial. Humans are still required to review large portions of AI-generated outputs directly, rather than operating primarily through high-level supervision and oversight.

Consequently, software engineering's long experience with testing, verification, and quality engineering is increasingly becoming the conceptual template for verification design in other AI-intensive domains. A key challenge is therefore: how do we adapt software engineering verification methodologies to AI coscientist and AI cowork systems?

As illustrated in Figure 2, across both AI-assisted software engineering and AIware engineering, two broad categories of tasks are emerging: solving tasks and verifying tasks. Solving tasks are concerned with generating behaviors and outputs. These include designing architectures, training models, generating code, constructing workflows, producing decisions, and executing actions. Such tasks are increasingly AI-driven, inexpensive, scalable, and amenable to automation. Verifying tasks, in contrast, are concerned with determining whether generated behaviors satisfy intended goals, constraints, specifications, or safety requirements. In domains such as AI cowork and AI coscientist, verification remains highly dependent on human

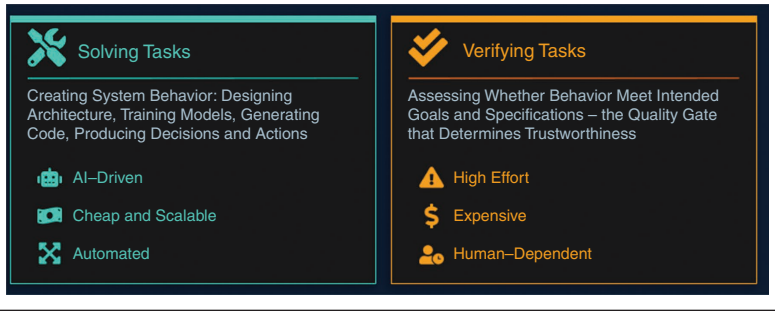



**FIGURE 2.** Two key types of tasks in AI engineering.

expertise and contextual judgment, making it expensive, difficult to scale, and often the dominant bottleneck.

In industry, it is often assumed that simply having a human in the loop guarantees effective oversight. However, this is often not the case. Placing a human reviewer at the end of a workflow to check final outputs, or inserting human review at every step, does not automatically ensure better outcomes. Human oversight itself should be treated as a safeguard that must be evaluated rather than assumed to be effective by default. In some settings, AI alone may outperform both humans assisted by AI support and humans without AI support.[3] This suggests that inserting humans into the loop does not necessarily lead to better decisions. The key question is therefore not whether humans should be included, but where and how they should be allocated within the workflow. Humans should be selectively assigned to tasks characterized by high uncertainty, deep contextual dependence, ethical judgment, or tasks that cannot yet be automatically verified, rather than being indiscriminately inserted throughout the entire process.

The central bottleneck in AI engineering is therefore no longer producing behaviors but verifying them. As Andrej Karpathy observed: "Software 1.0 easily automates what you can specify. Software 2.0 easily automates what you can verify."[4]

## Verification-Driven AI Engineering

Verification-driven AI engineering is a paradigm in which verifiability is treated as a first-class design objective in the engineering of AI systems, with problems deliberately formulated and decomposed to enable scalable and automated verification.[5] Rather than simply inserting AI into existing workflows and asking humans to review AI-generated outputs, verification-driven AI engineering advocates redesigning both the workflow and the verification layer around scalable verification.[6] The central idea is to explicitly separate tasks that can be automatically verified from tasks that are difficult to verify automatically. In this approach, automated verification becomes a primary objective in workflow design, while only a small subset of tasks requiring deep judgment, contextual understanding, or physical-world experimentation (e.g., wet-lab experiments) are escalated to humans. Human involvement is therefore selectively allocated to tasks characterized by high uncertainty and requiring rich contextual reasoning.

> Humans are still required to review large portions of AI-generated outputs directly, rather than operating primarily through high-level supervision and oversight.

We summarize five core design principles for scalable verification.

**First, decompose problems around verifiable behavioral claims.** Problems should be decomposed not only into functional components, but into explicit and testable behavioral claims that can be independently verified. Tasks that admit automated verification should be isolated wherever possible to maximize scalability and verification efficiency.

**Second, explicitly separate solving tasks and verifying tasks.** This enables exploitation of solve-verify asymmetry. Some problems are difficult to solve but easy to verify once a candidate solution is produced. Integer factorization is a canonical example: finding the factors of a large number can be computationally difficult, but verifying the solution is straightforward by multiplying the factors together. AI engineering workflows should therefore deliberately identify and exploit opportunities where verification is substantially easier than problem solving itself.

**Third, design verification across phases and levels.** Failures can emerge at different stages and levels of the system. Verification should therefore span both predeployment and postdeployment phases. Predeployment verification may include static analysis, testing, benchmark evaluation, simulation, and formal verification. Postdeployment verification may include runtime monitoring, behavioral verifiers, runtime simulation, policy enforcement, and anomaly detection. Verification should also operate at multiple levels. Case-level verification focuses on individual reasoning traces, generated code artifacts, or actions, while system-level verification considers long-term system behavior across executions, including safety, robustness, and policy

compliance over time. More broadly, AIware systems may require several complementary forms of verification, including verification of outputs, verification of workflows and execution processes, verification of adaptation dynamics, and assessment of long-term behavioral tendencies. These forms of verification may rely on substantially different engineering mechanisms, ranging from formal analysis and testing to runtime monitoring, simulation, calibration, and human oversight.

**Fourth, design for stackable and incremental verification.** Once a solution or component has been verified, its verification result should become reusable and composable. Verified artifacts should function as trusted building blocks that can be incrementally assembled into larger and more complex systems without requiring complete reverification from scratch.

**Fifth, allocate verification actors explicitly.** Verification responsibilities should be deliberately distributed across automated systems, runtime monitors, AI verifiers, simulation environments, and human experts. Some verification tasks can be fully automated, while others require human judgment. Human roles should be carefully designed to avoid degradation into ineffective rubber-stamping behavior.

There is a deeper engineering question: *what exactly is the stable object being verified*? In classical software engineering, verification is typically attached to relatively stable artifacts, such as code, interfaces, specifications, or test suites. In AIware systems, however, behavior may increasingly emerge from adaptive interactions among models, memory, retrieval, orchestration policies, external tools, and runtime feedback loops. In this setting, the "system" may partly consist of the adaptive process itself rather than a fixed artifact. Verification therefore increasingly concerns not only static outputs or components, but also evolving workflows, interaction dynamics, and long-term behavioral tendencies.

Verification in AIware systems may operate at multiple levels, including verification of outputs, workflows, adaptation dynamics, and longer-term behavioral patterns, each requiring different abstractions and evaluation mechanisms.

## The Articles in This Special Issue

This special issue brings together articles that examine the emerging engineering challenges of agentic systems across architecture, requirements engineering, testing, evaluation, human interaction, and domain deployment. Although the contributions span different application areas and technical concerns, they collectively reflect a broader transition in software engineering: system behavior is increasingly shaped not only by software code, but also by runtime reasoning, agent coordination, memory, tool use, and interaction with humans and external environments. This transition also reinforces the need for a verification-driven approach to AI engineering, where verification is treated not as a downstream testing activity, but as a first-class concern throughout the system lifecycle.

The articles in this special issue approach the same transition from complementary angles. Two articles help map the field by providing both conceptual and operational perspectives on the rapid emergence of agentic systems in practice. Ruscio et al. provide a taxonomy of emerging frameworks for engineering LLM-based multiagent systems.[A1] Their analysis shows that assembling agents is only one part of the problem: framework support for monitoring, performance evaluation, and quality assessment remains limited. Complementing this framework-oriented perspective, Su et al. examine practitioner adoption of AI agents through an analysis of industrial conference talks and practitioner discussions,[A2] which grounds the discussion in reported industrial practices, architectural choices, and deployment experiences.

The special issue also explores how agentic systems reshape collaborative and human-intensive engineering workflows. Sami et al. examine agentic support for requirements engineering through MARARE, a real-time multiagent system that converts live team meetings into draft product vision statements and feature descriptions.[A3] The work highlights how agents can support communication-intensive workflows where meaning and intent emerge dynamically through discussion and negotiation.

For the AI-assisted software engineering perspective, Donato et al. evaluate role-based multiagent code generation for repository-scale software engineering tasks.[A4] The results suggest that while multiagent approaches can improve coherence relative to individual agents, substantial challenges remain around integration, long-range context management, and repository-wide consistency.

Shi and Kristensson examine the adoption of LLM agents in software testing through a user-centered framework grounded in empirical studies of software professionals.[A5] Their framework considers factors such as interaction modality,

intervention effectiveness, disruption to existing practices, efficiency, and long-term risks. The article highlights that successful adoption of testing agents depends not only on model capability but also on how agentic systems integrate into established engineering workflows, organizational practices, and human review processes.

Arifin et al. examine agentic architectures for adaptive software vulnerability management through AgenticVM, a multiagent framework that integrates rule-based components, learned models, and LLM-driven agents across vulnerability detection, assessment, prioritization, and remediation workflows.[A6] Their architecture combines orchestration logic, confidence-gated prediction, schema validation, and human-in-the-loop safeguards to reduce alert fatigue while preserving operational control and accountability.

da Cruz et al. examine agentic systems in smart homes through Environmental Command Language Intelligent Processing System (ECLIPS), a two-layer modular architecture for smart homes that integrates LLM agents as a semantic interaction layer while preserving existing predictive models and automation logic.[A7] Their architectural approach demonstrates how conversational intelligence can augment existing systems without replacing established deterministic infrastructure. More broadly, the work reflects a growing architectural pattern in agentic systems: intelligence is distributed across models, traditional software components or on-demand generated code, external tools, and rather than residing entirely within a single model, creating new verification challenges across system boundaries.

## The Future of AIware Engineering from a Verification Perspective

The emerging AI technology stack is rapidly evolving beyond foundation models alone. As shown in Figure 3, from the bottom up, the stack includes data centers and energy infrastructure, hardware and compute, pretraining, post-training, inference-time system building, domain applications, trusted infrastructure, and verification and control. Among these, post-training, inference-time system building, domain applications, trusted infrastructure, verification and control layers are becoming particularly important for the software/AI engineering community.

Cross-cutting the entire stack is the need for trusted digital and AI infrastructure, including identity management, cybersecurity, secure data governance, and sovereign data management. More important is the verification and control layer, which focuses on ensuring trustworthiness through multilayer and multistage verification mechanisms, runtime monitoring, and guardrails.

At the post-training layer, the primary challenge is verifying open-ended behaviors, such as reasoning quality, instruction following, alignment, and safety. Existing evaluation approaches still rely heavily on static benchmarks, which are limited to anticipated failure modes and are vulnerable to benchmark overfitting. Moreover, correct outputs do not necessarily imply correct reasoning processes. Future AI engineering practices will likely move beyond leaderboard-style evaluation toward continuous, scenario-based, and adversarial evaluation pipelines that actively shape and improve system behavior over time. Synthetic oversight using AI-based monitors, structured reasoning traces, and task-specific verification tools may become increasingly important components of scalable oversight.

At the inference-time and AI system-building layer, behavior increasingly emerges not from isolated model outputs, but from the interaction of many system-level components. This introduces new verification challenges associated with complex multistep workflows and failure propagation across interconnected components. In many systems, runtime execution increasingly behaves like an ongoing organizational learning process in which behavior continuously evolves through interaction with memory, tools, retrieval systems, and verifier feedback. Verifying only final outputs becomes insufficient because intermediate failures may remain hidden while still producing plausible results. As a result, scalable verification requires structured execution traces, multilayer safeguards, runtime monitoring, and architectural decisions that move critical reasoning and decision-making from opaque model weights into more observable and controllable system components, such as tools, generated code, and explicit workflows.

At the domain application layer, AI systems increasingly operate in scientific and industrial settings such as health care, energy, agriculture, minerals, and biosecurity. In these domains, AI-generated hypotheses, plans, and decisions must

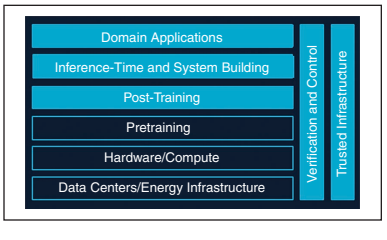
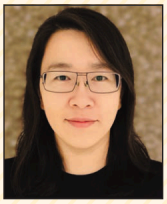



**FIGURE 3.** AI technology stack.

## ABOUT THE AUTHORS



**QINGHUA LU** is a senior principal research scientist and the group leader of Software Systems Research Group at CSIRO, Marsfield, NSW 2122, Australia. Contact her at qinghua.lu@csiro.au.



**AHMED E. HASSAN** is the Canada Research Chair and the NSERC/Research in Motion Industrial Research Chair in software engineering at Queen's University, Kingston, K7L 3N6, Canada. He is a Fellow of IEEE. Contact him at ahmed@queensu.ca.



**DAVID LO** is a professor in computer science and the director of the information and systems cluster with the School of Computing and Information Systems, Singapore Management University, Singapore 188065, Singapore. Contact him at davidlo@smu.edu.sg.



**THOMAS ZIMMERMANN** is Chancellor's Professor and Bren Chair at the University of California, Irvine, CA 92697, USA. Contact him at tzimmer@uci.edu.


ultimately be validated against physical laws, domain constraints, operational requirements, and expert knowledge. A major engineering challenge is separating scalable verification tasks, such as structural consistency and constraint satisfaction, from tasks requiring deep contextual judgment and human expertise. Physics-based simulation, digital twins, and world models are likely to play an increasingly important role as verification mechanisms for validating the plausibility and feasibility of AI-generated outputs. The integration of simulation and generative AI may also provide new pathways for building systems with trustworthiness-by-design.

The software engineering community is uniquely positioned to contribute to AI transformation. Decades of experience in testing, formal methods, verification, runtime monitoring, safety engineering, and quality assurance provide strong foundations for addressing the verification challenges emerging in AIware systems. The next frontier is extending these principles beyond software generation into broader domains, such as AI cowork and AI coscientist systems, where scalable verification may become one of the defining engineering challenges of the AI era. At the same time, future AIware engineering will likely require integrating verification with calibration, resilience engineering, economic/resource constraints, governance processes, and sociotechnical adaption mechanisms.

### Appendix: Related Articles


A1. D. Di Ruscio, P. T. Nguyen, C. Di Sipio, R. Rubei, and J. Di Rocco, "Engineering LLM-based multiagent systems: A taxonomy of emerging frameworks," *IEEE Softw.*, vol. 43, no. 5, pp. 30–37, Sep./Oct. 2026, doi: 10.1109/MS.2026.3694089.

A2. R. Su et al., "Making sense of AI agents hype: Adoption, architectures, and takeaways from practitioners," *IEEE Softw.*, vol. 43, no. 5, pp. 38–47, Sep./Oct. 2026, doi: 10.1109/MS.2026.3680431.

A3. M. A. Sami et al., "From meetings to product vision: An agentic approach with MARARE," *IEEE Softw.*, vol. 43, no. 5, pp. 48–55, Sep./Oct. 2026, doi: 10.1109/MS.2026.3702383.

A4. B. Donato, N. Hagar-Dent, A. Worsnop, L. Mariani, and V. Terragni, "An evaluation of role-based multiagent code generation on repository-scale problems," *IEEE Softw.*, vol. 43, no. 5, pp. 56–64, Sep./Oct. 2026, doi: 10.1109/MS.10.1109/MS.2026.3701516.

A5. B. Shi and P. O. Kristensson, "Adopting large language model agents in software testing: A user-centered framework," *IEEE Softw.*, vol. 43, no. 5, pp. 65–73, Sep./Oct. 2026, doi: 10.1109/MS.2026.3696850.

A6. A. Arifin, H. Ahmad, Y. Zhang, and D. Goel, "AgenticVM: Agentic

AI for adaptive software vulnerability management," *IEEE Softw.*, vol. 43, no. 5, pp. 74–81, Sep./Oct. 2026, doi: 10.1109/MS.2026.3691090.

A7. D. T. M. da Cruz et al., "From silent automation to conversational intelligence: Empowering smart homes with large language model-based agents," *IEEE Softw.*, vol. 43, no. 5, pp. 82–90, Sep./Oct. 2026, doi: 10.1109/MS.2026.3693949.

## References

1. L. Bass, Q. Lu, I. Weber, and L. Zhu, *Engineering AI Systems: Architecture and DevOps Essentials*. Addison-Wesley Professional, 2025.
2. A. E. Hassan et al., "Rethinking software engineering in the foundation model era: A curated catalogue of challenges in the development of trustworthy FMware," in *Proc. 32nd ACM Int. Conf. Found. Softw. Eng.*, 2024, pp. 294–305, doi: 10.1145/3663529.3663849.
3. D. McDuff et al., "Towards accurate differential diagnosis with large language models," *Nature*, vol. 642, pp. 451–457, Apr. 2025.
4. "Verifiability," *Karpathy*, Nov. 17, 2025. [Online]. Available: https://karpathy.bearblog.dev/verifiability/
5. L. Zhu and Q. Lu, "Verifiability-first AI engineering in the era of AIware: A conceptual framework, design principles, and architectural patterns for scalable verification." Jan. 2026, doi: 10.2139/ssrn.6031534. Available: SSRN: https://ssrn.com/abstract=6031534
6. Q. Lu, L. Zhu, S. Ma, H. Power, and R. Speight, "Verification-driven AI engineering: Workflows and reference architecture," 2026, doi: 10.2139/ssrn.6430979. [Online]. Available: SSRN: https://ssrn.com/abstract=6430979




*Digital Object Identifier 10.1109/MS.2026.3721539*